

SKILL-WEEK6

Task1:

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
UW PICO 5.09 File: job_list.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    int running;
};

struct Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, char command[])
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);

    jobs[job_count].running = 1;

    job_count++;
    next_job_id++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result > 0)
            {
                jobs[i].running = 0;
            }
        }
    }
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```
UW PICO 5.09 File: job_list.c Modified
jobs[job_count].pid = pid;
strcpy(jobs[job_count].command, command);

jobs[job_count].running = 1;

job_count++;
next_job_id++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            int status;

            pid_t result = waitpid(
                jobs[i].pid,
                &status,
                WNOHANG
            );

            if (result > 0)
            {
                jobs[i].running = 0;
            }
        }
    }
}

void list_jobs()
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No jobs\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            printf("[%d] %d Running %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
        else
        {
            printf("[%d] %d Done %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
    }
}

[?] Get Help      [?] WriteOut    [?] Read File    [?] Prev Pg     [?] Cut Text     [?] Cur Pos
[?] Exit          [?] Justify      [?] Where is    [?] Next Pg     [?] UnCut Text   [?] To Spell
```

```
UW PICO 5.09 File: job_list.c Modified
}

void list_jobs()
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No jobs\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            printf("[%d] %d Running %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
        else
        {
            printf("[%d] %d Done %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
    }
}

void remove_completed_jobs()
{
    int new_count = 0;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            jobs[new_count] = jobs[i];
            new_count++;
        }
    }

    job_count = new_count;
}

int main()
{
    char command[100];

    while (1)
    {
        update_jobs();

        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
    }
}

[?] Get Help      [?] WriteOut    [?] Read File    [?] Prev Pg     [?] Cut Text     [?] Cur Pos
[?] Exit          [?] Justify      [?] Where is    [?] Next Pg     [?] UnCut Text   [?] To Spell
```

UW PICO 5.09File: job_list.cModified

```
void remove_completed_jobs()
{
    int new_count = 0;
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            jobs[new_count] = jobs[i];
            new_count++;
        }
    }
    job_count = new_count;
}

int main()
{
    char command[100];
    while (1)
    {
        update_jobs();
        printf("myshell> ");
        fflush(stdout);
        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';
        if (strcmp(command, "exit") == 0)
        {
            break;
        }
        if (strcmp(command, "jobs") == 0)
        {
            list_jobs();
            remove_completed_jobs();
            continue;
        }
        int background = 0;
        int length = strlen(command);
        if (length > 0 && command[length - 1] == '&')
        {
            background = 1;
            command[length - 1] = '\0';
            while (strlen(command) > 0 &&
                  command[strlen(command) - 1] == ' ')
            {
                command[strlen(command) - 1] = '\0';
            }
        }
        Get Help
        Exit
        WriteOut
        Justify
        Read File
        Where is
        Prev Pg
        Next Pg
        Cut Text
        UnCut Text
        Cur Pos
        To Spell
    }
```

UW PICO 5.09File: job_list.c

```
        command[strcspn(command, "\n")] = '\0';
        if (strcmp(command, "exit") == 0)
        {
            break;
        }
        if (strcmp(command, "jobs") == 0)
        {
            list_jobs();
            remove_completed_jobs();
            continue;
        }
        int background = 0;
        int length = strlen(command);
        if (length > 0 && command[length - 1] == '&')
        {
            background = 1;
            command[length - 1] = '\0';
            while (strlen(command) > 0 &&
                  command[strlen(command) - 1] == ' ')
            {
                command[strlen(command) - 1] = '\0';
            }
        }
        char original_command[100];
        strcpy(original_command, command);
        char *args[10];
        int count = 0;
        char *token = strtok(command, " ");
        while (token != NULL && count < 9)
        {
            args[count] = token;
            count++;
            token = strtok(NULL, " ");
        }
        args[count] = NULL;
        if (count == 0)
        {
            continue;
        }
        pid_t pid = fork();
        if (pid == -1)
```

```
char original_command[100];

strcpy(original_command, command);

char *args[10];

int count = 0;

char *token = strtok(command, " ");

while (token != NULL && count < 9)
{
    args[count] = token;
    count++;

    token = strtok(NULL, " ");
}

args[count] = NULL;

if (count == 0)
{
    continue;
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    continue;
}

if (pid == 0)
{
    execvp(args[0], args);

    perror("exec");
    exit(1);
}

if (background)
{
    add_job(pid, original_command);

    printf("[%d] %d Running %s\n",
           next_job_id - 1,
           pid,
           original_command);
}
else
{
    waitpid(pid, NULL, 0);
}
}

return 0;
}
```

⌘ Get Help
⌘ Exit

⌘ WriteOut
⌘ Justify

⌘ Read File
⌘ Where is

⌘ Prev Pg
⌘ Next Pg

⌘ Cut Text
⌘ UnCut Text

⌘ Cur Pos
⌘ To Spell

```
Last login: Sat Sep 19 08:25:08 on console
(vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop/os
vishalkarnati@VISHALS-MacBook-Air os % mkdir week9skill
vishalkarnati@VISHALS-MacBook-Air os % nano job_list.c
vishalkarnati@VISHALS-MacBook-Air os % gcc job_list.c -o job_list
vishalkarnati@VISHALS-MacBook-Air os % ./job_list
myshell> sleep 20 &
[1] 2302 Running sleep 20
myshell> sleep 30 &
[2] 2303 Running sleep 30
myshell> jobs
[1] 2302 Running sleep 20
[2] 2303 Running sleep 30
myshell> jobs
[1] 2302 Done sleep 20
[2] 2303 Running sleep 30
myshell> exit
vishalkarnati@VISHALS-MacBook-Air os %
```

Task2:

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
UW PICO 5.09 File: foreground.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    int running;
};

struct Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, char command[])
{
    jobs[job_count].job_id = next_job_id;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);

    jobs[job_count].running = 1;

    printf("[%d] %d Running %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           jobs[job_count].command);

    job_count++;
    next_job_id++;
}

void list_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].running)
        {
            printf("[%d] %d Running %s\n",
                   jobs[i].job_id,
                   jobs[i].pid,
                   jobs[i].command);
        }
    }
}

void foreground_job(int job_id)
{
    int index = -1;

```

```
void foreground_job(int job_id)
{
    int index = -1;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id &&
            jobs[i].running)
        {
            index = i;
            break;
        }
    }

    if (index == -1)
    {
        printf("Job not found\n");
        return;
    }

    pid_t pid = jobs[index].pid;

    printf("Bringing job [%d] to foreground\n", job_id);

    /*
     * Give the terminal to the job.
     */
    tcsetpgrp(STDIN_FILENO, pid);

    /*
     * Resume the process if it was stopped.
     */
    kill(pid, SIGCONT);
}
```

```

/*
 * Wait for the foreground job.
 */
int status;

waitpid(pid, &status, WUNTRACED);

/*
 * Take terminal control back.
 */
tcsetpgrp(STDIN_FILENO, getpgrp());

/*
 * Update job state.
 */
if (WIFEXITED(status))
{
    jobs[index].running = 0;

    printf("Job [%d] completed\n", job_id);
}
else if (WIFSIGNALED(status))
{
    jobs[index].running = 0;

    printf("Job [%d] terminated\n", job_id);
}
else if (WIFSTOPPED(status))
{
    printf("Job [%d] stopped\n", job_id);
}
}

int main()
{
    /*
     * Ignore terminal-related signals in the shell.
     */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGSTP, SIG_IGN);

    char command[100];

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "jobs") == 0)
        {
            if (strcmp(command, "jobs") == 0)
            {
                list_jobs();
                continue;
            }

            if (strncmp(command, "fg ", 3) == 0)
            {
                int job_id;

                if (sscanf(command + 3, "%d", &job_id) == 1)
                {
                    foreground_job(job_id);
                }
                else
                {
                    printf("Usage: fg <job_id>\n");
                }
            }

            continue;
        }

        int background = 0;
        int length = strlen(command);

        if (length > 0 && command[length - 1] == '&')
        {
            background = 1;
            command[length - 1] = '\0';
        }
    }
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```

else if (WIFSTOPPED(status))
{
    printf("Job [%d] stopped\n", job_id);
}
}

int main()
{
    /*
     * Ignore terminal-related signals in the shell.
     */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);
    signal(SIGSTP, SIG_IGN);

    char command[100];

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }

        if (strcmp(command, "jobs") == 0)
        {
            if (strcmp(command, "jobs") == 0)
            {
                list_jobs();
                continue;
            }

            if (strncmp(command, "fg ", 3) == 0)
            {
                int job_id;

                if (sscanf(command + 3, "%d", &job_id) == 1)
                {
                    foreground_job(job_id);
                }
                else
                {
                    printf("Usage: fg <job_id>\n");
                }
            }

            continue;
        }

        int background = 0;
        int length = strlen(command);

        if (length > 0 && command[length - 1] == '&')
        {
            background = 1;
            command[length - 1] = '\0';
        }
    }
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```

if (strcmp(command, "jobs") == 0)
{
    list_jobs();
    continue;
}

if (strncmp(command, "fg ", 3) == 0)
{
    int job_id;

    if (sscanf(command + 3, "%d", &job_id) == 1)
    {
        foreground_job(job_id);
    }
    else
    {
        printf("Usage: fg <job_id>\n");
    }
}

continue;

int background = 0;
int length = strlen(command);

if (length > 0 && command[length - 1] == '&')
{
    background = 1;
    command[length - 1] = '\0';
}
}
}

```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

```

if (length > 0 && command[length - 1] == '&')
{
    background = 1;
    command[length - 1] = '\\0';

    while (strlen(command) > 0 &&
           command[strlen(command) - 1] == ' ')
    {
        command[strlen(command) - 1] = '\\0';
    }
}

char original_command[100];
strcpy(original_command, command);

char *args[10];

int count = 0;

char *token = strtok(command, " ");

while (token != NULL && count < 9)
{
    args[count] = token;
    count++;

    token = strtok(NULL, " ");
}

args[count] = NULL;

if (count == 0)
{
    continue;
}

```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text
 ⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ Cur Pos
 ⌘ To Spell

```

if (count == 0)
{
    continue;
}

pid_t pid = fork();

if (pid == -1)
{
    perror("fork");
    continue;
}

if (pid == 0)
{
    /*
     * Put the child into its own process group.
     */
    setpgid(0, 0);

    /*
     * Restore default signal behavior.
     */
    signal(SIGTTOU, SIG_DFL);
    signal(SIGTTIN, SIG_DFL);
    signal(SIGTSTP, SIG_DFL);

    execvp(args[0], args);

    perror("exec");
    exit(1);
}

```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text
 ⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ Cur Pos
 ⌘ To Spell

```

/*
 * Parent puts child into its own process group.
 */
setpgid(pid, pid);

if (background)
{
    add_job(pid, original_command);
}
else
{
    /*
     * Give terminal to foreground process.
     */
    tcsetpgrp(STDIN_FILENO, pid);

    int status;

    waitpid(pid, &status, WUNTRACED);

    /*
     * Shell gets terminal back.
     */
    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFSTOPPED(status))
    {
        add_job(pid, original_command);
    }
}

```

⌘ Get Help ⌘ WriteOut ⌘ Read File ⌘ Prev Pg ⌘ Cut Text
 ⌘ Exit ⌘ Justify ⌘ Where is ⌘ Next Pg ⌘ UnCut Text ⌘ Cur Pos
 ⌘ To Spell

```
        if (WIFSTOPPED(status))
        {
            add_job(pid, original_command);
        }
    }
}

return 0;
}
```

Get Help
Exit

WriteOut
Justify

Read File
Where is

Prev Pg
Next Pg

Cut Text
UnCut Text

Cur Pos
To Spell

```
(vishalkarnati@VISHALS-MacBook-Air os % cd week9skill
(vishalkarnati@VISHALS-MacBook-Air week9skill % nano foreground.c
(vishalkarnati@VISHALS-MacBook-Air week9skill % gcc foreground.c -o foreground
(vishalkarnati@VISHALS-MacBook-Air week9skill % ./foreground
myshell> sleep 20 &
[1] 2496 Running sleep 20
myshell> jobs
[1] 2496 Running sleep 20
myshell> fg 1
Bringing job [1] to foreground
Job [1] completed
myshell>
```

Task3:

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior

```
UW PICO 5.09 File: signal_analysis.c Modified
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received (Ctrl+C)\n");
    }
    else if (sig == SIGTSTP)
    {
        printf("\nSIGTSTP received (Ctrl+Z)\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received\n");
    }
}

int main()
{
    struct sigaction sa;

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTSTP, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);

    printf("Signal analysis program started\n");

    printf("PID      : %d\n", getpid());
    printf("Parent   : %d\n", getppid());
    printf("PGID    : %d\n", getpgrp());

    printf("\nWaiting for signals...\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}
```

Get Help
Exit
WriteOut
Justify
Read File
Where is
Prev Pg
Next Pg
Cut Text
UnCut Text
Cur Pos
To Spell

```
~/Desktop/os/week9skill --zsh ~/Desktop/os/week9skill --zsh
vishalkarnati@VISHALS-MacBook-Air week9skill % nano signal_analysis.c
vishalkarnati@VISHALS-MacBook-Air week9skill % gcc signal_analysis.c -o signal_analysis
vishalkarnati@VISHALS-MacBook-Air week9skill % ./signal_analysis
Signal analysis program started
PID      : 1628
Parent   : 1553
PGID    : 1628

Waiting for signals...
^C
SIGINT received (Ctrl+C)
^Z
SIGTSTP received (Ctrl+Z)

SIGUSR1 received
^Zsh: quit ./signal_analysis
vishalkarnati@VISHALS-MacBook-Air week9skill %
```

//signal masking

```
~/Desktop/os/week9skill -- nano signal_analysis.c                                ~/Desktop/os/week9skill -- -zsh
UW PICO 5.09                                                                    File: signal_analysis.c                                                           Modified

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received (Ctrl+C)\n");
    }
    else if (sig == SIGTSTP)
    {
        printf("\nSIGTSTP received (Ctrl+Z)\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received\n");
    }
}

int main()
{
    struct sigaction sa;

    sa.sa_handler = handle_signal;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTSTP, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);

    printf("Signal analysis program started\n");

    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("PGID : %d\n", getpgid());

    sigset_t mask;

    sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1);

    printf("\nBlocking SIGUSR1 for 10 seconds...\n");

    sigprocmask(SIG_BLOCK, &mask, NULL);

    for (int i = 10; i > 0; i--)
    {
        printf("%d seconds remaining...\n", i);
        sleep(1);
    }

    printf("\nUnblocking SIGUSR1...\n");

    sigprocmask(SIG_UNBLOCK, &mask, NULL);
```

```
~/Desktop/os/week9skill -- nano signal_analysis.c                                ~/Desktop/os/week9skill -- -zsh
UW PICO 5.09                                                                    File: signal_analysis.c                                                           Modified

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTSTP, &sa, NULL);
    sigaction(SIGUSR1, &sa, NULL);

    printf("Signal analysis program started\n");

    printf("PID : %d\n", getpid());
    printf("PPID : %d\n", getppid());
    printf("PGID : %d\n", getpgid());

    sigset_t mask;

    sigemptyset(&mask);
    sigaddset(&mask, SIGUSR1);

    printf("\nBlocking SIGUSR1 for 10 seconds...\n");

    sigprocmask(SIG_BLOCK, &mask, NULL);

    for (int i = 10; i > 0; i--)
    {
        printf("%d seconds remaining...\n", i);
        sleep(1);
    }

    printf("\nUnblocking SIGUSR1...\n");

    sigprocmask(SIG_UNBLOCK, &mask, NULL);

    printf("SIGUSR1 is now unblocked\n");

    printf("\nWaiting for signals...\n");

    while (1)
    {
        sleep(1);
    }

    return 0;
}
```

```

vishalkarnati@VISHALS-MacBook-Air week9skill % nano signal_analysis.c
vishalkarnati@VISHALS-MacBook-Air week9skill % gcc signal_analysis.c -o signal_analysis
vishalkarnati@VISHALS-MacBook-Air week9skill % ./signal_analysis
Signal analysis program started
PID : 1649
PPID : 1553
PGID : 1649

Blocking SIGUSR1 for 10 seconds...
10 seconds remaining...
9 seconds remaining...
8 seconds remaining...
7 seconds remaining...
6 seconds remaining...
5 seconds remaining...
4 seconds remaining...
3 seconds remaining...
2 seconds remaining...
1 seconds remaining...

Unblocking SIGUSR1...
SIGUSR1 is now unblocked

Waiting for signals...
SIGUSR1 received
^Zsh: quit      ./signal_analysis
vishalkarnati@VISHALS-MacBook-Air week9skill %

```

```

vishalkarnati@VISHALS-MacBook-Air ~ % cd ~/Desktop/os/week9skill
vishalkarnati@VISHALS-MacBook-Air week9skill % kill -SIGUSR1 1620
vishalkarnati@VISHALS-MacBook-Air week9skill % kill -SIGUSR1 1649
vishalkarnati@VISHALS-MacBook-Air week9skill %

```

Task4:

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```

~/Desktop/os/week9skill -- nano resume_jobs.c
~/Desktop/os/week9skill -- -zsh
UW PICO 5.09
File: resume_jobs.c
Modified

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_JOBS 10

struct Job
{
    int job_id;
    pid_t pid;
    char command[100];
    int state;
};

#define RUNNING 1
#define STOPPED 2
#define DONE 3

struct Job jobs[MAX_JOBS];

int job_count = 0;
int next_job_id = 1;

void add_job(pid_t pid, char command[], int state)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);

    jobs[job_count].state = state;

    printf("[%d] %s %s\n",
           jobs[job_count].job_id,
           jobs[job_count].pid,
           state == RUNNING ? "Running" : "Stopped",
           jobs[job_count].command);

    job_count++;
    next_job_id++;
}

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == DONE)

```

```

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == DONE)
        {
            continue;
        }

        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED
        );

        if (result == jobs[i].pid)
        {
            if (WIFSTOPPED(status))
            {
                jobs[i].state = STOPPED;
            }
            else if (WIFEXITED(status) ||
                     WIFSIGNALED(status))
            {
                jobs[i].state = DONE;
            }
        }
    }
}

```

```

void list_jobs()
{
    update_jobs();

    if (job_count == 0)
    {
        printf("No jobs\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state == RUNNING)
        {
            printf("[%d] %d Running %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
        else if (jobs[i].state == STOPPED)
        {
            printf("[%d] %d Stopped %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].command);
        }
    }
}

```

```

    else
    {
        printf("[%d] %d Done %s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].command);
    }
}
}

```

```

void resume_job(int job_id)
{
    int index = -1;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].job_id == job_id)
        {
            index = i;
            break;
        }
    }

    if (index == -1)
    {
        printf("Job not found\n");
        return;
    }

    if (jobs[index].state != STOPPED)
    {
        printf("Job is not stopped\n");
        return;
    }
}

```

⌘ Get Help
⌘ WriteOut
⌘ Read File
⌘ Prev Pg
⌘ Cut Text
⌘ Cur Pos

⌘ Exit
⌘ Justify
⌘ Where Is
⌘ Next Pg
⌘ UnCut Text
⌘ To Spell

```

printf("Resuming job [%d]\n", job_id);

if (kill(jobs[index].pid, SIGCONT) == -1)
{
    perror("SIGCONT");
    return;
}

jobs[index].state = RUNNING;

printf("[%d] %d Running %s\n",
    jobs[index].job_id,
    jobs[index].pid,
    jobs[index].command);
}

```

```

void remove_done_jobs()
{
    int new_count = 0;

    for (int i = 0; i < job_count; i++)
    {
        if (jobs[i].state != DONE)
        {
            jobs[new_count] = jobs[i];
            new_count++;
        }
    }
}

```

⌘ Get Help
⌘ WriteOut
⌘ Read File
⌘ Prev Pg
⌘ Cut Text
⌘ Cur Pos

⌘ Exit
⌘ Justify
⌘ Where Is
⌘ Next Pg
⌘ UnCut Text
⌘ To Spell

```

    }
    job_count = new_count;
}

int main()
{
    char command[100];

    /*
     * Ignore terminal job-control signals in the shell.
     */
    signal(SIGTTOU, SIG_IGN);
    signal(SIGTTIN, SIG_IGN);

    while (1)
    {
        update_jobs();

        printf("myshell> ");
        fflush(stdout);

        fgets(command, sizeof(command), stdin);
        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            break;
        }
    }
}

```

⌘ Get Help
⌘ WriteOut
⌘ Read File
⌘ Prev Pg
⌘ Cut Text
⌘ Cur Pos

⌘ Exit
⌘ Justify
⌘ Where is
⌘ Next Pg
⌘ UnCut Text
⌘ To Spell

```

    if (strcmp(command, "jobs") == 0)
    {
        list_jobs();
        continue;
    }

    if (strncmp(command, "resume ", 7) == 0)
    {
        int job_id;

        if (sscanf(command + 7, "%d", &job_id) == 1)
        {
            resume_job(job_id);
        }
        else
        {
            printf("Usage: resume <job_id>\n");
        }
        continue;
    }

    if (strncmp(command, "start ", 6) == 0)
    {
        char *program = command + 6;
        pid_t pid = fork();
    }
}

```

⌘ Get Help
⌘ WriteOut
⌘ Read File
⌘ Prev Pg
⌘ Cut Text
⌘ Cur Pos

⌘ Exit
⌘ Justify
⌘ Where is
⌘ Next Pg
⌘ UnCut Text
⌘ To Spell

```

    if (pid == -1)
    {
        perror("fork");
        continue;
    }

    if (pid == 0)
    {
        setpgid(0, 0);

        execvp(program, program, NULL);

        perror("exec");
        exit(1);
    }

    setpgid(pid, pid);

    /*
     * Give terminal to child.
     */
    tcsetpgrp(STDIN_FILENO, pid);

    int status;

    waitpid(pid, &status, WUNTRACED);

    /*
     * Take terminal back.
     */
}

```

⌘ Get Help
⌘ WriteOut
⌘ Read File
⌘ Prev Pg
⌘ Cut Text
⌘ Cur Pos

⌘ Exit
⌘ Justify
⌘ Where is
⌘ Next Pg
⌘ UnCut Text
⌘ To Spell

```

    /*
     * Take terminal back.
     */
    tcsetpgrp(STDIN_FILENO, getpgrp());

    if (WIFSTOPPED(status))
    {
        add_job(pid, program, STOPPED);
    }
    else
    {
        printf("Process completed\n");
    }
    continue;
}

printf("Unknown command\n");
}

remove_done_jobs();

return 0;
}

```

⌘ Get Help
⌘ WriteOut
⌘ Read File
⌘ Prev Pg
⌘ Cut Text
⌘ Cur Pos

⌘ Exit
⌘ Justify
⌘ Where is
⌘ Next Pg
⌘ UnCut Text
⌘ To Spell

```
vishalkarnati@VISHALS-MacBook-Air week9skill % nano resume_jobs.c
vishalkarnati@VISHALS-MacBook-Air week9skill % gcc resume_jobs.c -o resume_jobs
vishalkarnati@VISHALS-MacBook-Air week9skill % ./resume_jobs
myshell> start sleep
usage: sleep number[unit] [...]
Unit can be 's' (seconds, the default), m (minutes), h (hours), or d (days).
Process completed
myshell> ^Z
```